

TAREA 6 TEMA 2

Autenticación basada en Certificado con Linux

BASTIONADO DE REDES Y SISTEMAS

ÍNDICE

1. Introducción.....	3
2. Preparación del entorno.....	3
2.1. Ubuntu Server 22.04 o superior.....	3
2.2. Comprobación del estado inicial del servidor SSH.....	3
El servidor debe tener instalado el servicio SSH.....	3
Lo comprobé con:.....	3
2.3. Copias de seguridad del fichero sshd_config.....	4
3. Creación de la Autoridad Certificadora (CA).....	4
3.1. Generación de la CA de usuario.....	5
3.2. Generación de la CA de host (opcional).....	6
4. Emisión del certificado de usuario.....	6
4.1. Generación de par de claves del usuario.....	6
4.2. Firma del certificado con la CA.....	7
4.3. Análisis del certificado generado (ssh-keygen -L).....	8
5. Configuración del servidor SSH.....	9
5.1. Importación de la CA de usuarios.....	9
5.2. Modificación de sshd_config.....	10
5.3. Reinicio del servicio y comprobaciones.....	11
6. Pruebas de conexión con certificado.....	12
6.1. Intento de conexión sin certificado → fallo esperado.....	12
6.2. Conexión con certificado → éxito.....	13
6.3. Logs y análisis con ssh -vvv.....	13
7. Revocación de certificados.....	14
7.1. ¿Qué es una Key Revocation List (KRL)?.....	14
7.2. Creación de la lista KRL.....	14
7.3. Configuración en sshd_config.....	15
7.4. Revocación del certificado del usuario.....	15
7.5. Prueba de acceso tras la revocación.....	15
8. Conclusiones personales.....	16



1. Introducción

En esta práctica he trabajado la autenticación en SSH utilizando certificados, un sistema que va un paso más allá de las claves públicas tradicionales. La idea principal es crear una Autoridad Certificadora (CA) que sea la encargada de firmar los certificados de los usuarios y validar si pueden o no acceder al servidor. De esta forma, el acceso no depende solo de una clave pública copiada en el servidor, sino de un certificado que tiene una fecha de validez, una identidad y unas restricciones que podemos controlar.

El objetivo de este ejercicio es entender cómo funciona este tipo de autenticación, configurarlo desde cero en un Ubuntu Server y comprobar que el sistema reconoce y valida correctamente los certificados. Además, también he visto cómo se puede revocar el acceso de un usuario sin necesidad de borrar su cuenta, que es algo importante cuando se quiere gestionar permisos de manera profesional.

Al final, esta práctica sirve para ver cómo se usan realmente los certificados en entornos de seguridad y por qué son una opción más segura y más flexible que las claves SSH normales. También me ha ayudado a entender mejor la configuración interna de SSH y cómo se controlan los accesos en un servidor Linux.

2. Preparación del entorno

Antes de empezar con la configuración de la autenticación SSH mediante certificados, es necesario preparar el entorno para evitar errores y asegurarnos de que todo funciona correctamente. Para esta práctica he utilizado los siguientes requisitos:

2.1. Ubuntu Server 22.04 o superior

Es importante trabajar sobre una versión actualizada de Ubuntu Server, ya que las versiones modernas de OpenSSH incluyen soporte completo para certificados, revocación y validación de CA.

2.2. Comprobación del estado inicial del servidor SSH

El servidor debe tener instalado el servicio SSH.

Lo comprobé con:

```
sudo systemctl status ssh
```



Con esto confirmé que el servicio estaba en ejecución y que no había errores previos que pudieran afectar a la configuración.

Si el servicio no estuviera instalado, se podría instalar con:

```
sudo apt install openssh-server
```

Aunque en mi caso ya estaba operativo.

```
paula@paula-VirtualBox:~/bunkerweb-practica$ sudo systemctl status ssh
[sudo] contraseña para paula:
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/lib/systemd/system/ssh.service; enabled; vendor preset: enabled)
   Active: active (running) since Tue 2025-12-02 16:25:06 CET; 2h 25min ago
     Docs: man:sshd(8)
           man:sshd_config(5)
    Main PID: 811 (sshd)
      Tasks: 1 (limit: 4594)
     Memory: 3.0M
        CPU: 19ms
    CGroup: /system.slice/ssh.service
            └─811 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

dic 02 16:25:06 paula-VirtualBox systemd[1]: Starting OpenBSD Secure Shell server...
dic 02 16:25:06 paula-VirtualBox sshd[811]: Server listening on 0.0.0.0 port 22.
dic 02 16:25:06 paula-VirtualBox sshd[811]: Server listening on :: port 22.
dic 02 16:25:06 paula-VirtualBox systemd[1]: Started OpenBSD Secure Shell server.
paula@paula-VirtualBox:~/bunkerweb-practica$
```

2.3. Copias de seguridad del fichero sshd_config

Como este archivo es el que controla toda la configuración de SSH, lo primero fue hacer una copia de seguridad para poder restaurarla en caso de errores:

```
sudo cp /etc/ssh/sshd_config /etc/ssh/sshd_config.backup
```

De esta forma puedo volver al estado original si algo falla durante la configuración. Además, es una buena práctica siempre que se va a tocar un servicio crítico del sistema.

3. Creación de la Autoridad Certificadora (CA)

Para poder usar autenticación SSH mediante certificados, lo primero es crear una Autoridad Certificadora (CA). Esta CA será la encargada de firmar los certificados de los usuarios y, opcionalmente, también los certificados de los servidores. Su función es muy similar a la de una CA en HTTPS, pero aplicada a SSH.



En esta práctica he creado una CA específica para usuarios, que será la que valide quién puede acceder al servidor.

3.1. Generación de la CA de usuario

Para generar la CA, utilicé el comando `ssh-keygen`, creando un par de claves:

- una clave privada (*que debe mantenerse protegida y nunca se copia al servidor*),
- y una clave pública (*que sí se subirá al servidor como CA confiable*).

El comando utilizado fue:

```
ssh-keygen -t rsa -b 4096 -f user_ca -C "CA-usuarios"
```

```
com tg-backup
paula@paula-VirtualBox:~/bunkerweb-practica$ ssh-keygen -t rsa -b 4096 -f user_ca -C "CA-usuarios"
Generating public/private rsa key pair.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in user_ca
Your public key has been saved in user_ca.pub
The key fingerprint is:
SHA256:Y55KE7wD4loA0DV0FA511mN8pgtExlxC20wtXfDu6s CA-usuarios
The key's randomart image is:
+----[RSA 4096]-----+
|.oB.. ooB*oo|
|.* o . +*=.o.|
|+ o . .Bo +|
|.. . =.o. o|
|. . . o So .|
| o . . = o .|
| o = o .|
| o . + .|
|. . . E..|
+-----[SHA256]-----+
paula@paula-VirtualBox:~/bunkerweb-practica$
```

Este mensaje confirma que se ha iniciado la creación del par de claves de la CA (clave privada y clave pública).

Cuando se pidió la passphrase, la dejé vacía para simplificar el uso durante la práctica.

Tras completar el proceso, los archivos generados fueron:

- `user_ca` (clave privada de la CA)
- `user_ca.pub` (clave pública de la CA)



Comprobé su existencia con:

```
+-----[SHA256]-----+
paula@paula-VirtualBox:~/bunkerweb-practica$ ls -l user_ca*
-rw----- 1 paula paula 3369 dic  2 18:54 user_ca
-rw-r--r-- 1 paula paula  737 dic  2 18:54 user_ca.pub
paula@paula-VirtualBox:~/bunkerweb-practica$
```

3.2. Generación de la CA de host (opcional)

Esta CA sirve para validar automáticamente la identidad del servidor al que nos conectamos, evitando los típicos avisos de "The authenticity of host can't be established".

El comando sería:

```
ssh-keygen -t rsa -b 4096 -f host_ca -C "CA-hosts"
```

Esto generaría:

- host_ca → clave privada de la CA para hosts
- host_ca.pub → clave pública para firmar certificados de servidor

En mi caso, la dejo documentada por si la quisiera implementar más adelante.

4. Emisión del certificado de usuario

Una vez creada la Autoridad Certificadora (CA), el siguiente paso es generar el certificado que permitirá al usuario autenticarse en el servidor mediante SSH. Para ello, primero se crea un par de claves del usuario y después se firma la clave pública con la CA.

4.1. Generación de par de claves del usuario

El usuario necesita su propio par de claves (privada y pública), que serán la base del certificado.

Para crearlas utilicé:

```
ssh-keygen -t rsa -b 4096 -f user_key -C "usuario-cert"
```



```

Ayuda paula@paula-VirtualBox:~/bunkerweb-practica$ ssh-keygen -t rsa -b 4096 -f user_key -C "u
suario-cert"
Generating public/private rsa key pair.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in user_key
Your public key has been saved in user_key.pub
The key fingerprint is:
SHA256:TS1qAYKCHXoVj9gE0r7IGQL8mln9Mq/Iz+odIC+MU18 usuario-cert
The key's randomart image is:
+---[RSA 4096]---+
|+oo+=..          |
|+++= + . . .     |
|oo+ + . . o .    |
|.o.o . = .       |
|ooOo  E S .      |
|+B+..+ o         |
|oo ...+          |
|.o + ..          |
| .o=.            |
+---[SHA256]-----+
paula@paula-VirtualBox:~/bunkerweb-practica$

```

Esto genera dos archivos:

- user_key → clave privada del usuario
- user_key.pub → clave pública del usuario

Ambos se guardan en el directorio donde ejecutes el comando.

Verifiqué su creación con:

`ls -l user_key*`

```

+---[SHA256]-----+
paula@paula-VirtualBox:~/bunkerweb-practica$ ls -l user_key*
-rw----- 1 paula paula 3381 dic  2 18:59 user_key
-rw-r--r-- 1 paula paula  738 dic  2 18:59 user_key.pub
paula@paula-VirtualBox:~/bunkerweb-practica$

```

4.2. Firma del certificado con la CA

Una vez generadas las claves, firmé la clave pública con la CA de usuarios (user_ca).

El comando usado fue:

`ssh-keygen -s user_ca -I paula-cert -n paula -V +52w user_key.pub`



-s user_ca → clave privada de la CA que firma el certificado

-I paula-cert → identificador del certificado

-n paula → principal, es decir, el usuario del sistema al que se le permite acceder

-V +52w → validez del certificado (52 semanas en este caso)

Este proceso genera el archivo:

- user_key-cert.pub → el certificado SSH del usuario

```
paula@paula-VirtualBox:~/bunkerweb-practica$ ssh-keygen -s user_ca -I paula-cert -n paula -V +52w user_key.pub
Signed user key user_key-cert.pub: id "paula-cert" serial 0 for paula valid from 2025-12-02T19:00:00 to 2026-12-01T19:01:17
paula@paula-VirtualBox:~/bunkerweb-practica$
```

4.3. Análisis del certificado generado (ssh-keygen -L)

Para ver el contenido y validar que la configuración era correcta, usé:

`ssh-keygen -L -f user_key-cert.pub`

```
paula@paula-VirtualBox:~/bunkerweb-practica$ ssh-keygen -L -f user_key-cert.pub
user_key-cert.pub:
  Type: ssh-rsa-cert-v01@openssh.com user certificate
  Public key: RSA-CERT SHA256:TS1qAYKCHXoVj9gE0r7IGQL8mLn9Mq/Iz+odIC+MU18
  Signing CA: RSA SHA256:Y55KE7wD4loAODV0FA511mN8pgtExlxC20wtXfDu6s (using rsa-sha2-512)
  Key ID: "paula-cert"
  Serial: 0
  Valid: from 2025-12-02T19:00:00 to 2026-12-01T19:01:17
  Principals:
    paula
  Critical Options: (none)
  Extensions:
    permit-X11-forwarding
    permit-agent-forwarding
    permit-port-forwarding
    permit-pty
    permit-user-rc
paula@paula-VirtualBox:~/bunkerweb-practica$
```



Este comando muestra:

- Identidad del certificado
- Principals permitidos
- Fecha de emisión y de caducidad
- CA que lo ha firmado
- Restricciones aplicadas

Es una buena forma de asegurar que el certificado está bien configurado antes de usarlo.

5. Configuración del servidor SSH

Una vez generado el certificado del usuario, es necesario configurar el servidor SSH para que pueda validar esos certificados. Para ello, el servidor debe conocer la clave pública de la CA que hemos creado y habilitar las opciones que permiten este tipo de autenticación.

5.1. Importación de la CA de usuarios

En este paso transferí el archivo `user_ca.pub` (clave pública de la CA) al servidor Ubuntu.

Puede hacerse con `scp`, con copia manual o con cualquier método, pero lo importante es colocarlo en una ubicación segura, normalmente:

```
sudo mv user_ca.pub /etc/ssh/
```

```
sudo chmod 644 /etc/ssh/user_ca.pub
```

Después verifiqué que el archivo existía con:

```
ls -l /etc/ssh/user_ca.pub
```

```
paula@paula-VirtualBox:~/bunkerweb-practica$ sudo mv user_ca.pub /etc/ssh/  
sudo chmod 644 /etc/ssh/user_ca.pub  
paula@paula-VirtualBox:~/bunkerweb-practica$ ls -l /etc/ssh/user_ca.pub  
-rw-r--r-- 1 paula paula 737 dic  2 18:54 /etc/ssh/user_ca.pub  
paula@paula-VirtualBox:~/bunkerweb-practica$
```



5.2. Modificación de sshd_config

Una vez que la clave pública de la CA (user_ca.pub) estaba en su ubicación correcta, edité el archivo de configuración del servicio SSH para activar la autenticación mediante certificados.

Abrí el archivo con:

```
sudo nano /etc/ssh/sshd_config
```

Dentro del archivo añadí la siguiente línea:

```
TrustedUserCAKeys /etc/ssh/user_ca.pub
```

Esta directiva le indica al servidor que debe confiar en cualquier certificado firmado por la CA que yo he creado.

Además, verifiqué que estas líneas estuvieran habilitadas:

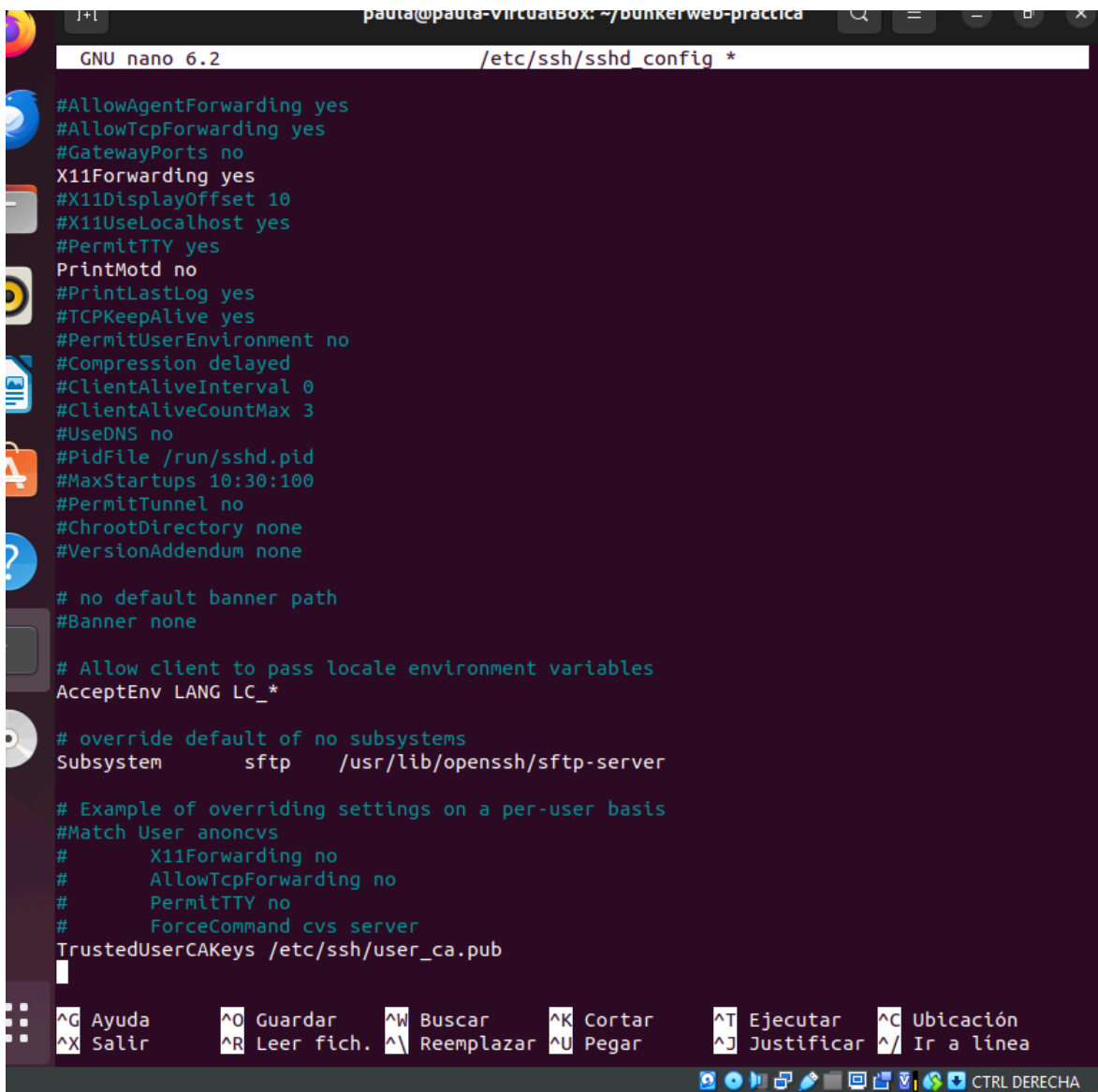
```
PubkeyAuthentication yes
```

```
AuthorizedKeysFile .ssh/authorized_keys
```

Con esto me aseguro de que SSH permita autenticación por clave pública y mantenga el sistema tradicional sin conflicto con los certificados.

Guardé los cambios y cerré el archivo.





The screenshot shows a terminal window with the title bar 'paula@paula-virtualBox: ~/bunkerweb-practica'. The terminal is running the GNU nano 6.2 editor, editing the file /etc/ssh/sshd_config. The file content is as follows:

```
#AllowAgentForwarding yes
#AllowTcpForwarding yes
#GatewayPorts no
X11Forwarding yes
#X11DisplayOffset 10
#X11UseLocalhost yes
#PermitTTY yes
PrintMotd no
#PrintLastLog yes
#TCPKeepAlive yes
#PermitUserEnvironment no
#Compression delayed
#ClientAliveInterval 0
#ClientAliveCountMax 3
#UseDNS no
#PidFile /run/sshd.pid
#MaxStartups 10:30:100
#PermitTunnel no
#ChrootDirectory none
#VersionAddendum none

# no default banner path
#Banner none

# Allow client to pass locale environment variables
AcceptEnv LANG LC_*

# override default of no subsystems
Subsystem        sftp        /usr/lib/openssh/sftp-server

# Example of overriding settings on a per-user basis
#Match User anoncvs
#    X11Forwarding no
#    AllowTcpForwarding no
#    PermitTTY no
#    ForceCommand cvs server
TrustedUserCAKeys /etc/ssh/user_ca.pub
```

At the bottom of the terminal, there is a navigation bar with the following shortcuts: ^G Ayuda, ^X Salir, ^O Guardar, ^R Leer fich., ^W Buscar, ^N Reemplazar, ^K Cortar, ^U Pegar, ^T Ejecutar, ^C Ubicación, ^J Justificar, ^_ Ir a línea. The bottom status bar shows 'CTRL DERECHA'.

5.3. Reinicio del servicio y comprobaciones

Después de modificar el archivo `sshd_config`, es necesario reiniciar el servicio SSH para que los cambios entren en vigor. Para ello utilicé el siguiente comando:

```
sudo systemctl restart ssh
```

Una vez reiniciado, comprobé que el servicio se había cargado correctamente y que no existían errores de sintaxis en la configuración. Para verificarlo ejecuté:

```
sudo systemctl status ssh
```



```
paula@paula-VirtualBox:~/bunkerweb-practica$ sudo systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/lib/systemd/system/ssh.service; enabled; vendor preset: enabled)
   Active: active (running) since Tue 2025-12-02 19:12:21 CET; 4s ago
     Docs: man:sshd(8)
           man:sshd_config(5)
   Process: 43669 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
    Main PID: 43670 (sshd)
      Tasks: 1 (limit: 4594)
     Memory: 1.8M
        CPU: 11ms
    CGroup: /system.slice/ssh.service
            └─43670 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

dic 02 19:12:21 paula-VirtualBox systemd[1]: Starting OpenBSD Secure Shell server...
dic 02 19:12:21 paula-VirtualBox sshd[43670]: Server listening on 0.0.0.0 port 22.
dic 02 19:12:21 paula-VirtualBox sshd[43670]: Server listening on :: port 22.
dic 02 19:12:21 paula-VirtualBox systemd[1]: Started OpenBSD Secure Shell server.
paula@paula-VirtualBox:~/bunkerweb-practica$
```

Al arrancar correctamente nos indica que la directiva *TrustedUserCAKeys* */etc/ssh/user_ca.pub* ha sido aceptada y el servidor está listo para validar certificados firmados por la CA configurada anteriormente.

6. Pruebas de conexión con certificado

Una vez configurado el servidor SSH para aceptar certificados firmados por la CA, realicé varias pruebas para verificar que el sistema funciona correctamente. En este apartado se recogen las comprobaciones realizadas tanto con clave tradicional como con el certificado generado.

6.1. Intento de conexión sin certificado → fallo esperado

Para asegurarme de que el servidor ya no permite cualquier clave pública y depende de la CA, intenté conectarme únicamente con la clave **sin** el certificado:

```
ssh -i user_key paula@127.0.0.1
```

En este caso, la conexión fue rechazada y el servidor respondió con un mensaje indicando que el método de autenticación falló, lo cual es el comportamiento esperado cuando la clave no está firmada mediante certificado.

Este paso confirma que el servidor no acepta claves sin certificado y que la autenticación basada en claves tradicionales está desactivado



6.2. Conexión con certificado → éxito

Con la clave privada y el certificado generado en la misma carpeta:

`ssh -i user_key paula@127.0.0.1`

```
Oct 02 19:55:50 paula-VirtualBox-systemd[1]: Started OpenSSH Secure Shell server.  
paula@paula-VirtualBox:~/bunkerweb-practica$ ssh -i user_key paula@127.0.0.1  
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-139-generic x86_64)  
  
* Documentation:  https://help.ubuntu.com  
* Management:    https://landscape.canonical.com  
* Support:        https://ubuntu.com/pro  
  
El mantenimiento de seguridad expandido para Applications está desactivado  
  
Se pueden aplicar 2 actualizaciones de forma inmediata.  
Para ver estas actualizaciones adicionales, ejecute: apt list --upgradable  
  
23 actualizaciones de seguridad adicionales se pueden aplicar con ESM Apps.  
Aprenda más sobre cómo activar el servicio ESM Apps at https://ubuntu.com/esm  
  
287 updates could not be installed automatically. For more details,  
see /var/log/unattended-upgrades/unattended-upgrades.log  
Last login: Thu Nov  6 18:51:25 2025  
paula@paula-VirtualBox:~$
```

La autenticación fue correcta sin necesidad de introducir contraseña.

6.3. Logs y análisis con `ssh -vvv`

Para comprobar en detalle cómo se estaba realizando la autenticación y confirmar que el servidor estaba aceptando correctamente el certificado firmado por la CA, utilicé el modo verbose de SSH. Este modo muestra paso a paso el proceso de autenticación y permite identificar si el cliente está ofreciendo el certificado, si el servidor lo reconoce y si lo acepta.

El comando utilizado fue:

`ssh -vvv -i user_key paula@127.0.0.1`

```
debug2: shell request accepted on channel 0  
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-139-generic x86_64)
```



7. Revocación de certificados

Una de las ventajas de utilizar autenticación SSH basada en certificados es que podemos invalidar el acceso de un usuario sin necesidad de borrar su cuenta del sistema, ni modificar `authorized_keys`, ni regenerar claves.

Esto se hace mediante las Listas de Revocación de Claves, conocidas como KRL (Key Revocation List).

El funcionamiento es sencillo: el servidor mantiene un fichero donde se guardan certificados revocados. Cada vez que un usuario intenta autenticarse, el servidor comprueba si su certificado aparece en la lista KRL. Si está revocado, deniega el acceso automáticamente.

7.1. ¿Qué es una Key Revocation List (KRL)?

Una KRL es un archivo que mantiene un listado de:

- Certificados revocados
- Claves públicas que ya no deben ser válidas
- Series, IDs o huellas de certificados no autorizados

Cuando el servidor SSH recibe un certificado, comprueba si está en la KRL. Si aparece ahí, se considera invalido y el acceso se bloquea al instante.

7.2. Creación de la lista KRL

Lo primero que hice fue crear el archivo vacío que actuará como lista de revocación:

```
sudo touch /etc/ssh/revoked_keys
```

Este archivo será administrado por el administrador del sistema y contendrá los certificados revocados.



7.3. Configuración en sshd_config

Para que el servidor SSH utilice este archivo, añadí la siguiente línea en:

```
sudo nano /etc/ssh/sshd_config
```

Y coloqué:

```
RevokedKeys /etc/ssh/revoked_keys
```

Con esto, SSH revisará este archivo cada vez que un usuario presente un certificado.

Apliqué los cambios reiniciando el servicio:

```
sudo systemctl restart ssh
```

7.4. Revocación del certificado del usuario

Para revocar el certificado generado anteriormente, utilicé el comando ssh-keygen con la opción -k, que permite actualizar una lista KRL.

Desde mi carpeta de trabajo ejecuté:

```
sudo ssh-keygen -k -f /etc/ssh/revoked_keys -s user_ca user_key-cert.pub
```

```
paula@paula-VirtualBox:~/bunkerweb-practica$ sudo ssh-keygen -k -f /etc/ssh/revoked_keys  
-s user_ca user_key-cert.pub  
Revoking from user_key-cert.pub  
paula@paula-VirtualBox:~/bunkerweb-practica$
```

7.5. Prueba de acceso tras la revocación

Una vez revocado, volví a intentar conectarme con:

```
ssh -i user_key paula@127.0.0.1
```



```
Revoking from user_key-cert.pub  
paula@paula-VirtualBox:~/bunkerweb-practica$ ssh -i user_key paula@127.0.0.1  
(paula@127.0.0.1) Verification code:
```

Aunque me pide la verification code, ahora el servidor ya no te acepta el certificado, no como antes, que te autenticabas **sin contraseña**, usando el certificado

8. Conclusiones personales

Esta práctica me ha parecido útil porque me ha permitido trabajar paso a paso con un sistema de autenticación SSH distinto al habitual. He configurado la CA, he generado y firmado certificados, y he comprobado cómo se utilizan realmente durante la conexión. También he podido ver cómo funciona la revocación mediante una lista KRL, que permite desactivar un certificado sin modificar la cuenta del usuario.

En general, ha sido una práctica directa y con una aplicación clara, que me ha servido para familiarizarme con este tipo de configuración y con las opciones que ofrece SSH para gestionar accesos de forma más controlada.

